

LIBRARY BOOK RECOMMENDATION SYSTEM

A Project Report on Design and Analysis of Algorithms

Submitted by:

Name: G. Hansini Reddy

G. Hasini

M. Truptirekha

V. Aishwarya

Roll Number: 25MVCSMR0034

25MVCSMR0038

25MVCSMR0085

25MVCSMR0088

Department: CSE – (AIML)

Year/Semester: 2nd / 2

University: Malla Reddy Technical Campus

Academic Year: 2026–2027

COVER PAGE

LIBRARY BOOK RECOMMENDATION SYSTEM

The **Library Book Recommendation System** is an algorithm-based library assistant developed to provide efficient book searching and related-book recommendations.

The project demonstrates important concepts from **Design and Analysis of Algorithms (DAA)**, particularly:

- Binary Search
- Sorting
- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Graph Data Structures

The system can help users search for books quickly and discover other books related to their interests.

Project Domain:

Library Management and Book Recommendation

Core Algorithms:

- Binary Search
- Sorting
- BFS
- DFS

Technologies Used:

- Python
- Flask
- SQLite
- React

ABSTRACT

2.1 Abstract

The Library Book Recommendation System is a digital library assistant designed to make book searching and book discovery easier and more efficient. In a large library, manually searching through a large collection of books can take considerable time. Users may also find it difficult to identify books that are related to their interests. This project addresses these problems by combining searching, sorting, graph traversal, and recommendation techniques.

The system uses **Binary Search** to efficiently search for books in a sorted collection. Instead of checking every book one by one, Binary Search repeatedly divides the search area into two parts and reduces the number of comparisons required.

Sorting is used to arrange books according to different attributes such as title, author, category, or borrowing count. Properly sorted data also supports efficient searching and organized presentation of results.

The project represents relationships between books using a **graph**. Each book is treated as a node, while relationships between books are represented using edges. Books can be connected because they belong to the same category, have the same author, or have similar borrowing patterns.

Breadth-First Search (BFS) and **Depth-First Search (DFS)** are then used to explore the book relationship graph. BFS explores related books level by level, while DFS explores one path deeply before moving to another path.

The proposed system can be implemented using Python for algorithms, Flask for backend development, SQLite for database management, and React for the frontend interface.

Overall, the project demonstrates how fundamental DAA concepts can be applied to a practical real-world problem. It provides both an efficient way to search for books and a method to discover related books through graph-based recommendations.

Keywords:

Library Management, Book Recommendation, Binary Search, Sorting, Graph, BFS, DFS, Python, Flask, SQLite, React, DAA

INTRODUCTION

3.1 Introduction

Libraries contain a large collection of books belonging to different subjects, categories, and authors. As the number of books increases, finding a particular book manually becomes difficult and time-consuming.

A digital library system can solve this problem by allowing users to search and organize books electronically. However, a useful library system should not only find a particular book but should also help users discover other books that may be related to their interests.

The **Library Book Recommendation System** is designed for this purpose. It combines fundamental algorithms from Design and Analysis of Algorithms with a digital library application.

The system uses Binary Search for efficient searching, Sorting for organizing book records, and graph traversal algorithms such as BFS and DFS for exploring relationships between books.

3.2 Need for the System

Suppose a library contains thousands of books. If a user wants to find a specific book, checking every book individually can require many comparisons.

Binary Search can reduce this searching effort when the data is sorted.

Similarly, after finding a book, the user may want to know:

- Which books belong to the same category?
- Which books are written by the same author?
- Which books are frequently borrowed?
- Which books are related to the selected book?

The recommendation component of the proposed system helps answer these questions.

3.3 Role of DAA

Design and Analysis of Algorithms helps in selecting efficient methods for solving computational problems.

In this project:

- **Sorting** organizes the library data.
- **Binary Search** finds books efficiently.
- **Graphs** represent relationships between books.
- **BFS** explores books level by level.
- **DFS** explores books deeply through connected paths.

Therefore, the project provides a practical demonstration of important DAA concepts.

PROBLEM STATEMENT, OBJECTIVES AND SCOPE

4.1 Problem Statement

The main problem addressed by this project is the difficulty of efficiently searching and discovering books in a large digital library.

A basic library system may allow users to search for books, but it may not provide an effective mechanism for recommending related books.

The proposed system aims to provide an algorithm-based library assistant that can:

1. Search for books efficiently.
2. Sort books according to different attributes.
3. Represent relationships between books.
4. Explore related books using BFS and DFS.
5. Recommend related books using available library information.
6. Display important algorithmic information such as comparisons and traversal steps.

4.2 Objectives

The main objectives are:

- To develop a digital library assistant.
- To implement Binary Search for efficient book searching.
- To sort library records.
- To represent books using suitable data structures.
- To represent relationships between books using graphs.
- To implement BFS.
- To implement DFS.
- To recommend related books.
- To provide a simple and user-friendly interface.

4.3 Scope

The scope of the project includes:

- Book searching
- Book sorting
- Book details
- Category-based recommendations
- Author-based recommendations
- Borrowing-pattern analysis
- Graph-based relationships
- BFS traversal
- DFS traversal
- Algorithm performance information

The project can later be extended into a complete library management system.

FEATURES AND FUNCTIONAL REQUIREMENTS

5.1 Major Features

1. Book Search

The user can enter the name of a book and search for it in the library.

Binary Search can be used when the book records are sorted according to title.

2. Book Sorting

Books can be sorted based on:

- Title
- Author
- Category
- Borrow count

3. Book Details

After selecting a book, the system can display:

- Book ID
- Title
- Author
- Category
- Borrow count
- Related books

4. Related Book Discovery

The system can identify books related to the selected book.

5. Recommendation

Recommendations can be generated using:

- Similar categories
- Same authors
- Borrowing patterns
- Graph relationships

6. Graph Representation

Books are represented as nodes and relationships between books are represented as edges.

7. BFS and DFS

BFS and DFS can be applied to explore the book graph.

8. Algorithm Statistics

The system can display information such as:

- Number of comparisons
- Search steps
- Traversal order
- Number of nodes visited
- Execution time, if measured

5.2 Functional Requirements

ID Functional Requirement

FR1 The system shall allow users to search for books.

FR2 The system shall allow users to sort book records.

FR3 The system shall display book information.

FR4 The system shall identify related books.

FR5 The system shall perform BFS traversal.

FR6 The system shall perform DFS traversal.

FR7 The system shall generate recommendations.

FR8 The system shall display algorithm-related information.

BINARY SEARCH AND SORTING

6.1 Binary Search

Binary Search is an efficient searching algorithm used on sorted data.

Instead of checking each element one by one, Binary Search compares the target with the middle element.

If the target is smaller, the left half is searched.

If the target is larger, the right half is searched.

This process continues until the element is found or the search range becomes empty.

6.2 Example

Suppose the sorted book titles are:

Atomic Habits

Harry Potter

Ikigai

The Alchemist

The Hobbit

If the user searches for **The Hobbit**, Binary Search checks the middle element first.

The search space is then reduced until the required book is found.

6.3 Basic Steps

1. Set the left and right boundaries.
2. Find the middle position.
3. Compare the middle book with the target.
4. If they are equal, return the book.
5. If the target is smaller, search the left half.
6. Otherwise, search the right half.
7. Repeat until the book is found or the range becomes empty.

6.4 Complexity

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$

6.5 Sorting

Sorting arranges book records in a particular order.

The system can sort books according to:

- Title
- Author
- Category
- Borrow count

Sorting is important because Binary Search requires sorted data.

6.6 Importance of Sorting

Sorting helps the system:

- Organize records.
- Improve searching.
- Display books clearly.
- Rank books.
- Perform range-based operations.

The exact sorting complexity depends on the sorting method implemented.

GRAPH REPRESENTATION AND BFS

7.1 Graph Representation

A graph is a data structure consisting of vertices and edges.

In this project:

- Each **book is a vertex/node**.
- A relationship between two books is an **edge**.

For example:

Harry Potter

/ \

/ \

The Hobbit ---- Fantasy Book

|

|

Related Book

Books can be connected because they have:

- Same category
- Same author
- Similar borrowing patterns
- Other defined relationships

7.2 Adjacency List

An adjacency list can represent the graph.

Example:

Harry Potter → The Hobbit, Fantasy Book

The Hobbit → Harry Potter, Fantasy Book

Fantasy Book → Harry Potter, The Hobbit

7.3 Breadth-First Search

Breadth-First Search, or BFS, is a graph traversal algorithm.

BFS explores nodes **level by level**.

It normally uses a **queue**.

BFS Steps

1. Select the starting book.
2. Mark it as visited.
3. Add it to the queue.
4. Remove a book from the queue.
5. Visit its unvisited connected books.
6. Add those books to the queue.
7. Continue until the queue becomes empty.

BFS Example

If:

A → B, C

B → D

C → E

Starting from A, BFS visits:

A → B → C → D → E

7.4 BFS Complexity

For a graph with V vertices and E edges:

Time Complexity = $O(V + E)$

BFS can be useful for finding nearby or minimum-hop related books.

DFS AND BFS VS DFS

8.1 Depth-First Search

Depth-First Search, or DFS, is another graph traversal algorithm.

DFS explores one path deeply before returning and exploring another path.

DFS can be implemented using:

- Recursion
- Stack

DFS Steps

1. Select a starting book.
2. Mark it as visited.
3. Visit an unvisited connected book.
4. Continue deeper into the graph.
5. Backtrack when there are no unvisited connected books.
6. Continue until all required nodes are explored.

Example

For:

$A \rightarrow B, C$

$B \rightarrow D$

$C \rightarrow E$

A possible DFS traversal is:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow E$

The exact traversal order can depend on the order of neighbors.

8.2 DFS Complexity

For V vertices and E edges:

Time Complexity = $O(V + E)$

8.3 BFS vs DFS

Feature	BFS	DFS
Full Form	Breadth-First Search	Depth-First Search
Main Structure	Queue	Stack/Recursion
Approach	Level by level	Deep first
Useful For	Nearby nodes	Deep exploration
Time Complexity	$O(V+E)$	$O(V+E)$
Memory	Can be larger for wide graphs	Can be smaller for some graphs
Project Use	Finding nearby related books	Exploring deeper relationships

Both algorithms are useful for demonstrating different methods of graph traversal.

RECOMMENDATION METHODOLOGY

9.1 Recommendation System

The recommendation component identifies books that may be related to the selected book.

The system can consider multiple factors.

1. Category Similarity

Books belonging to the same category can be recommended.

For example:

Selected Book: Harry Potter

Category: Fantasy

Possible Recommendations:

- The Hobbit
- Other Fantasy Books

2. Author Similarity

Books written by the same author can be considered related.

3. Borrowing Patterns

Frequently borrowed books can receive higher priority in the recommendation process.

4. Graph Relationships

The graph shows direct and indirect relationships between books.

BFS can be used to find books that are close to the selected book in the graph.

DFS can be used to explore deeper relationships.

9.2 Recommendation Workflow

User Selects Book



Find Book



Check Category



Check Author



Check Borrowing Pattern



Explore Book Graph



Apply BFS / DFS



Find Related Books



Generate Recommendations



Display Results

9.3 Example

If the user selects **Harry Potter**, the system may check:

- Fantasy category
- Author information
- Borrowing count
- Connected books in the graph

The system can then display a list of related books.

The recommendation process demonstrates how different data structures and algorithms can work together in a single application.

TECHNOLOGIES AND IMPLEMENTATION

10.1 Python

Python is used for implementing the main algorithms and application logic.

It provides simple syntax and useful data structures such as:

- Lists
- Dictionaries
- Sets
- Queues
- Stacks

Python can be used to implement Binary Search, Sorting, BFS, DFS, and recommendation logic.

10.2 Flask

Flask is used to develop the backend.

The backend can provide API endpoints such as:

/search

/books

/sort

/recommend

/bfs

/dfs

The frontend sends requests to the Flask backend, and Flask processes them.

10.3 SQLite

SQLite is a lightweight relational database.

It can store:

- Book information

- Author information
- Categories
- Borrow counts
- Book relationships

10.4 React

React can be used to develop the frontend interface.

The interface may contain:

- Search bar
- Book cards
- Sorting buttons
- Recommendation section
- Graph/traversal information

10.5 Implementation Modules

The project can be divided into the following modules:

Module 1 — Book Management

Stores and retrieves book information.

Module 2 — Search

Implements Binary Search.

Module 3 — Sorting

Sorts books based on selected criteria.

Module 4 — Graph

Creates relationships between books.

Module 5 — BFS

Explores the graph level by level.

Module 6 — DFS

Explores the graph deeply.

Module 7 — Recommendation

Combines book information and graph results to generate recommendations.

Module 8 — User Interface

Displays search results, recommendations, and algorithm information.

SYSTEM WORKFLOW, TESTING AND RESULTS

11.1 Complete System Workflow

User



React Interface



Search Request



Flask Backend



SQLite Database



Sort Book Records



Binary Search



Book Found



Graph Relationships



BFS / DFS



Recommendation Module



Recommended Books



Display Results

11.2 Testing

Testing is performed to verify whether the major functions of the system work correctly.

Test Cases

Test ID	Test Case	Expected Result
T01	Search existing book	Book should be displayed
T02	Search unavailable book	Not-found message should be displayed
T03	Sort by title	Books should appear alphabetically
T04	Perform BFS	Correct traversal should be displayed
T05	Perform DFS	Correct traversal should be displayed
T06	Generate recommendation	Related books should be displayed
T07	Search empty input	Appropriate validation message
T08	View book details	Correct book information displayed

11.3 Expected Results

The system should successfully:

- Search books.
- Sort books.
- Display book details.
- Represent book relationships.
- Perform BFS.
- Perform DFS.
- Generate related-book recommendations.

11.4 Algorithm Results

Binary Search should reduce the number of comparisons required compared with checking every record sequentially, provided that the data is sorted.

BFS should explore connected books level by level.

DFS should explore connected books deeply before backtracking.

Sorting should organize the records and make the collection easier to search and analyze.

PERFORMANCE, CONCLUSION, FUTURE SCOPE AND REFERENCES

12.1 Performance Analysis

The performance of the algorithms can be studied using metrics such as:

- Number of comparisons
- Execution time
- Number of nodes visited
- Number of edges explored
- Search steps

Algorithm Complexity

Algorithm	Time Complexity
Binary Search	$O(\log n)$
BFS	$O(V + E)$
DFS	$O(V + E)$
Sorting	Depends on sorting algorithm

The actual execution time can vary depending on the size of the dataset, hardware, programming implementation, and database operations.

12.2 Conclusion

The **Library Book Recommendation System** demonstrates how Design and Analysis of Algorithms can be applied to a practical library-related problem.

The project uses Binary Search to provide efficient searching of sorted book records. Sorting is used to organize the library collection according to different attributes.

The project also uses a graph to represent relationships between books. BFS and DFS are applied to explore these relationships and identify related books.

The recommendation component combines information such as category, author, borrowing patterns, and graph connections to provide useful book suggestions.

The project therefore demonstrates how multiple algorithms and data structures can work together to create a practical application. It also helps in understanding the

importance of algorithm selection, complexity analysis, graph representation, and efficient data processing.

12.3 Future Scope

The system can be improved in several ways:

1. **Machine Learning Recommendations**
Machine learning techniques can be used to provide more personalized recommendations.
2. **User Profiles**
Individual user reading history can be stored and analyzed.
3. **Personalized Recommendations**
Recommendations can be based on a user's previous searches and borrowing history.
4. **Real-Time Availability**
The system can display whether a book is currently available.
5. **Larger Database**
The system can be connected to a larger library database.
6. **Mobile Application**
A mobile application can be developed for easier access.
7. **Advanced Graph Analysis**
More advanced graph algorithms can be used to study relationships between books.
8. **Cloud Deployment**
The application can be deployed on a cloud platform.
9. **Real Library Integration**
The system can be integrated with an actual library management system.

12.4 References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, **Introduction to Algorithms**.
2. Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran, **Computer Algorithms**.
3. Python Documentation.
4. Flask Documentation.
5. SQLite Documentation.
6. React Documentation.
7. Standard academic resources on graph algorithms, searching, and sorting.

PROJECT SUMMARY

The Library Book Recommendation System combines **Binary Search, Sorting, Graphs, BFS, and DFS** to create an algorithm-based digital library assistant.

The major flow of the project is:

Search → Sort → Find Book → Explore Relationships → BFS/DFS → Recommend Books

The project demonstrates the practical application of DAA concepts while providing an efficient and user-friendly approach to library book discovery.

THANK YOU